

ORIGINAL INCIDENT RESEARCH

Advanced WordPress Malware Removal

PERSISTENT INFECTIONS,
REINFECTION, AND RECOVERY

SC 4.0.3 CASE STUDY

SELF-HEALING WORDPRESS MALWARE

FILES | DATABASE | MEMORY | BROWSERS

MD Pabel

A CASE STUDY ADAPTED FROM CHAPTER 16 OF THE COMPLETE BOOK

Persistent Infections, Reinfection, and Recovery

MD Pabel

First edition, version 1.0 - August 2026

Copyright (c) 2026 MD Pabel. All rights reserved.

You may download and share this field guide unchanged for noncommercial use. You may not sell it, alter it, remove the author attribution, or republish it as your own work without written permission.

This field guide is adapted from Chapter 16 of *WordPress Malware Removal for Developers & Site Owners*.

WordPress is a trademark of the WordPress Foundation. Product, service, and company names are used for identification and education. Their inclusion does not imply endorsement.

Important safety note

This publication is educational material, not legal, regulatory, insurance, or incident-response advice for every environment. Investigate only websites, accounts, and systems you own or are authorized to work on. Preserve evidence and backups before changing production data.

Treat recovered malware as hostile data. Do not execute a sample, include it from PHP, load it through WordPress, call `eval`, visit its endpoints, or contact its command-and-control infrastructure merely to see what happens. Use an isolated analysis environment and retain non-executable evidence.

Malicious destinations are defanged or omitted, sensitive information is redacted, and code is limited to defensive analysis and cleanup examples.

Contents

How to Use This Field Guide	3
SC 4.0.3: The WordPress Malware That Rebuilt Itself	5
Field Checklist	27
Continue with the Complete Book	30

How to Use This Field Guide

This is a practical case study for developers and freelancers who clean hacked WordPress sites and need to explain why malware sometimes returns after the obvious file has been deleted.

The case is unusual in scale but useful in method. SC 4.0.3 connected normal WordPress execution paths, PHP configuration, plugins, must-use plugins, drop-ins, a theme, ZIP archives, database records, process memory, scheduled behavior, administrator access, and a browser service worker. Any surviving recovery source could help restore another component.

You do not need to memorize its filenames. Names and paths change. Instead, follow the relationships: what executes first, where a clean-looking copy can be recovered, which database rows coordinate it, what access remains, and what event causes the infection to reappear.

What you will learn

- how to map a multi-layer WordPress persistence system before deleting it;
- how to decode Base64 and compressed PHP samples without executing them;
- how to confirm database persistence with read-only SQL;
- how to separate stable malware markers from site-specific candidates;
- how to clean files, database state, memory, credentials, browsers, and sibling sites in one maintenance window; and
- how to verify recovery by testing the triggers that previously restored the infection.

Who this guide is for

The examples assume you can inspect files and use phpMyAdmin or an equivalent database client. Optional shell and PowerShell examples are included, but the investigation does not depend on command-line access.

If this is an active client incident, read the whole guide before deleting anything. The order matters: preserve, contain, map, validate, remove, revoke, and verify.

Evidence and transparency

This case study is based on recovered files, matching decoded payload hashes, privacy-reviewed incident screenshots, and read-only database queries. It separates behavior proven by static code from behavior observed on the affected site. It also records what the evidence could not establish, including the original entry point and whether every recovered capability executed.

The analysis examples written for this guide are labeled as examples. They are not presented as code recovered from the malware. Complete executable payloads, live attacker destinations, credentials, cookies, and sensitive database values are not published.

The goal is not to delete one suspicious plugin. The goal is to remove every surviving writer and recovery source, then prove that the former restoration paths no longer work.

SC 4.0.3: The WordPress Malware That Rebuilt Itself

I encountered a WordPress infection that appeared to repair itself almost as quickly as its files were deleted.

The visible plugin used the name **Trace Scanner Lite**. Removing that plugin did not end the infection. The same payload also existed as a must-use plugin, inside WordPress drop-ins, at the end of a theme's `functions.php`, in hidden and visible PHP loaders, in ZIP archives, in WordPress database rows, and in process memory. The recovered code could even enroll an administrator's browser as another persistence layer.

The investigation follows one complex incident from the first directory listing through static decoding, database confirmation, coordinated eradication, and verification. The operating rule is simple: when malware comes back, find what writes, restores, downloads, or reactivates it.

The important lesson is not the fake plugin name. It is the relationship between the components.

Self-healing malware is not magical. It returns because at least one trusted execution path and at least one recovery source are still available.

What "self-healing" means in this case

The term describes behavior, not a claim that the malware is alive or invincible. The recovered components repeatedly checked for missing copies and could rebuild them from surviving sources.

The first useful evidence was the wp-content directory:

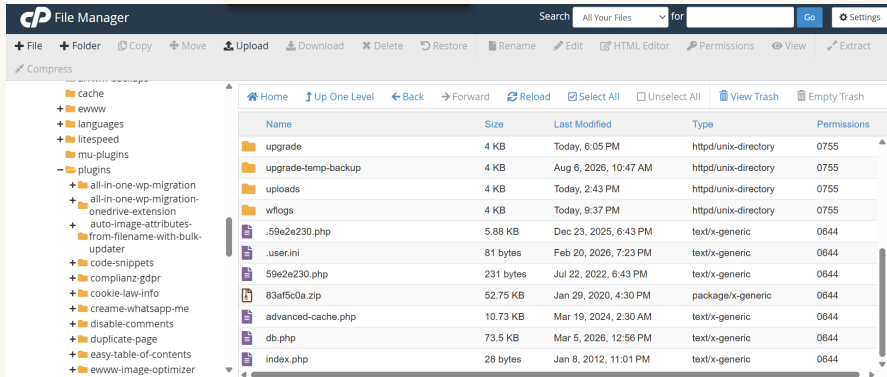
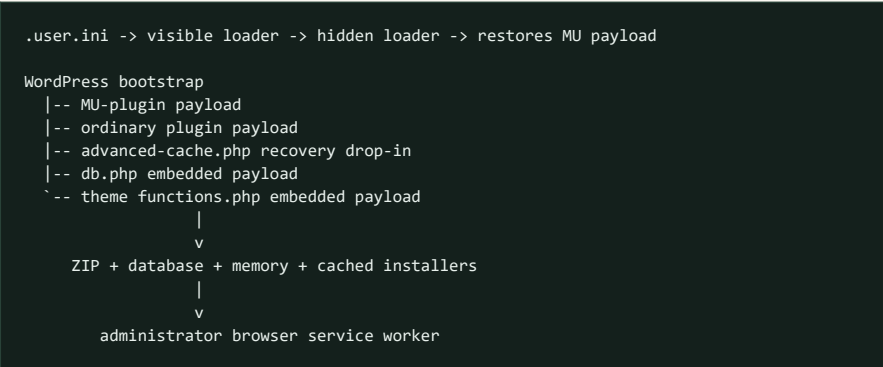


Figure 16.1. The same directory held `.user.ini`, a small visible loader, a larger hidden loader, a ZIP recovery archive, `advanced-cache.php`, and `db.php`. Seeing them together made a multi-component persistence system more likely, but their contents still had to be examined.

The visible `59e2e230.php` was a 231-byte wrapper. It included the larger dot-prefixed `.59e2e230.php`. On Unix-like systems, the leading dot makes a file easy to miss when hidden files are not displayed. The `.user.ini` file set `auto_prepend_file`, allowing PHP to load the wrapper before ordinary application code in the applicable directory scope.^[1]

WordPress then supplied more automatic execution paths. Top-level PHP files in `wp-content/mu-plugins` load automatically. `advanced-cache.php` and `db.php` are recognized WordPress drop-ins. The active theme loads its `functions.php` file. The infection used each legitimate mechanism for an unauthorized purpose.^[2]

The persistence architecture can be summarized like this:



This is a graph, not a single chain. Several nodes can restore another node, and the main payload can rebuild the supporting nodes. Deleting one file leaves the graph connected.

Build an evidence map before deleting anything

I recorded every observed component and the behavior proven by its recovered code. This prevents a suspicious filename from becoming a conclusion by itself.

Artifact	Behavior established in this case
.user.ini	Configured PHP to prepend the visible loader.
wp-content/59e2e230.php	Included the hidden loader.
wp-content/.59e2e230.php	Located wp-content and restored the MU payload from surviving sources.
mu-plugins/trace-scanner-lite.php	Automatically loaded primary payload and preferred persistence copy.
plugins/trace-scanner-lite/trace-scanner-lite.php	Redundant ordinary-plugin copy using an attacker-controlled identity.
advanced-cache.php	Recovered the payload from files, ZIP, database, or shared memory.
db.php	Contained a complete compressed copy of the primary payload.
active-theme functions.php	Contained another complete compressed payload.
83af5c0a.zip and copies	Stored the plugin directory and payload for restoration.
WordPress options and transients	Stored payloads, a manifest, recovery state, and coordination data.
System V shared memory	Supplied a process-resident payload backup when the PHP extension was available.

Artifact	Behavior established in this case
browser service worker	Could capture login submissions, alter admin responses, receive JavaScript, and attempt plugin reinstallation.

The plugin appeared in both WordPress plugin locations:

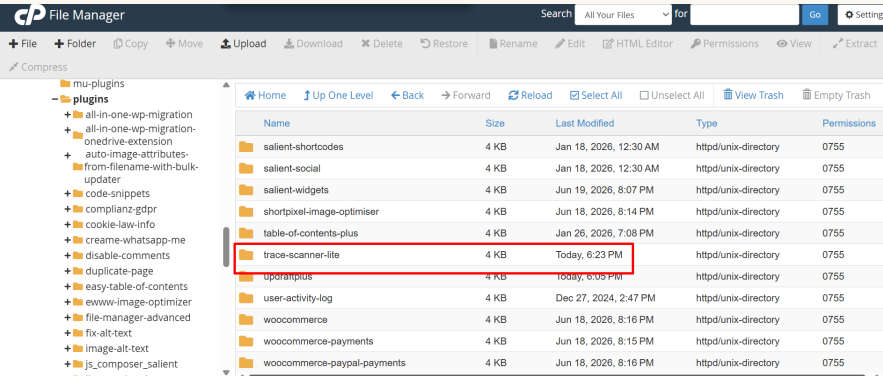


Figure 16.2. The ordinary plugin directory contained the alias observed on this site. A plugin header is attacker-controlled text, so the name is a pivot for this incident, not a universal family identifier.

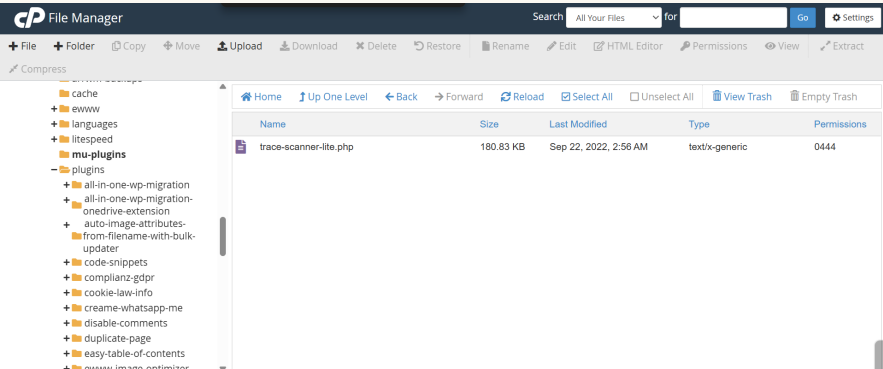


Figure 16.3. The MU-plugin copy was 180.83 KB and displayed permission mode 0444. The restrictive mode made casual editing harder, while the recovered code could change permissions when replacing its own files.

The normal and MU copies were not two unrelated infections. They were replicas in the same recovery system. The payload also filtered plugin lists, update data, Site Health results, and some file-manager results. Absence from the ordinary Plugins screen therefore did not prove absence from disk.

How I decoded the samples without executing them

Malware analysis becomes more dangerous when "decode" is confused with "run." I treated every recovered PHP file as untrusted data. I did not include it from PHP, load it through WordPress, call `eval`, visit its endpoints, or contact an Ethereum RPC provider.

The static workflow had five stages:

1. preserve the original files and calculate hashes;
2. inventory readable strings, markers, paths, hooks, and encoded regions;
3. reconstruct only the sample's literal string-substitution layer;
4. decode Base64 and decompress gzip data into files with a `.txt` suffix; and
5. compare the resulting bytes and hashes across recovery nodes.

Stage 1: preserve and hash first

I worked from copies in an isolated analysis directory. The originals stayed unchanged. On Windows, this command records a SHA-256 without loading the PHP:

```
Get-FileHash -Algorithm SHA256 -LiteralPath '.\captured-sample.php'
```

For a folder of evidence, the same operation can be repeated without executing any file:

```
Get-ChildItem -LiteralPath '.\evidence' -File |  
  Get-FileHash -Algorithm SHA256 |  
  Sort-Object Path
```

Hashes answer two different questions. A hash of the original file proves which captured object was analyzed. A hash of a decoded payload shows whether two differently wrapped artifacts contain identical bytes.

Stage 2: normalize only literal substitution

The recovered samples hid strings behind an indexed table and substitution functions. During the original analysis, the literal table and deterministic transformation were inspected without executing the PHP.

The following snippet is an illustrative defensive helper written for this book. It was not recovered from the malware. It shows how literal pairs transcribed from a specific sample could be applied while treating the captured PHP only as text:

```
import { readFileSync, writeFileSync } from "node:fs";

const source = readFileSync("captured-sample.php", "utf8");
const replacements = new Map([
  // Add only literal pairs recovered from the sample under analysis.
  // ["encoded-token", "decoded-token"],
]);

let normalized = source;
for (const [encoded, decoded] of replacements) {
  normalized = normalized.split(encoded).join(decoded);
}

writeFileSync("normalized-sample.php.txt", normalized, "utf8");
```

`captured-sample.php` is a placeholder filename, and the replacement map is intentionally empty. The example performs string replacement only. It does not implement PHP, call WordPress, fetch a URL, or evaluate the result. A real sample may use a different table or transformation, so the replacement map must come from that sample rather than from a copied online recipe.

The normalized text exposed stable markers and roles such as:

```
SC_ADV_BEGIN:4.0.3:
SC_DB_BEGIN:4.0.3:
SC_TH_BEGIN:4.0.3:
sc_payload_persistent
sc_persist_manifest
sc_cron_fetch
```

Stage 3: extract Base64 and decompress offline

The `db.php` implant and the block appended to the theme each held 71,804 Base64 characters. The decoded bytes began with the gzip signature. The following Node.js example looks for long Base64 regions, accepts only a gzip candidate, writes the decompressed content with a `.txt` extension, and prints its hash. It never executes the content.

```
import { readFileSync, writeFileSync } from "node:fs";
import { gunzipSync } from "node:zlib";
import { createHash } from "node:crypto";

const input = readFileSync(process.argv[2], "utf8");
const candidates = input.match(/[A-Za-z0-9+/{1000,}={0,2}/g) ?? [];

for (const [index, encoded] of candidates.entries()) {
  const packed = Buffer.from(encoded, "base64");
  const isGzip = packed[0] === 0x1f && packed[1] === 0x8b;
  if (!isGzip) continue;

  const decoded = gunzipSync(packed);
  const hash = createHash("sha256").update(decoded).digest("hex");
  const output = `decoded-${index}-${hash.slice(0, 12)}.php.txt`;

  writeFileSync(output, decoded);
  console.log(`${output}\t${decoded.length}\t${hash}`);
}
```

Run a decoder like this only in an isolated workspace, against a copied sample, with networking disabled. Do not rename the output to `.php`, place it under a web root, or open it through a PHP-enabled preview tool.

Stage 4: compare decoded payloads

The payload recovered from both wrappers was 185,171 bytes. Each produced this SHA-256:

```
8284d68274c0475118732110680e38d9da27b4173997c0db0d9f6e5283109554
```

That hash also matched the recovered `trace-scanner-lite.php` payload. This byte-for-byte equality is stronger than visual similarity. It proves that the drop-in, theme implant, and fake plugin were storing the same core payload.

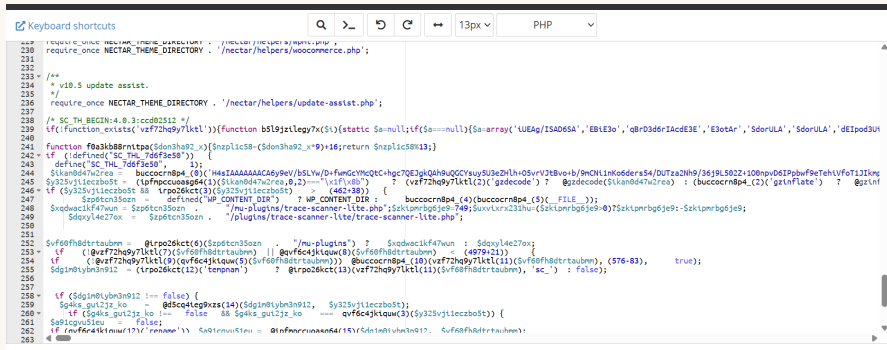


Figure 16.4. The attacker appended an `SC_TH_BEGIN:4.0.3` block after legitimate theme code. Decoding its embedded gzip data produced the exact hash of the primary payload.

This also changes the cleanup decision. Editing only the visible plugin is not enough. The theme must be restored from a trusted copy or carefully diffed so the complete marked implant is removed without destroying legitimate custom work.

Stage 5: inspect behavior as text

After normalization, I followed filesystem writes, WordPress hooks, database keys, scheduled events, service-worker templates, and network configuration as code paths. Capabilities were recorded only when the path was directly present in the recorded code.

For example, the service-worker template was 6,429 bytes with SHA-256:

```
7bafee00a010ed3db3521a89e8f5d50d2f677204c56995e99c7603768a9f0bd2
```

The template could intercept WordPress login submissions, receive JavaScript, alter admin navigations, and attempt plugin installation through an already authenticated browser. That finding came from static code, not from installing the service worker in a browser.

Recovery archives and site-specific names

The payload wrote ZIP backups under `wp-content`, the current uploads year and month, and a writable theme. Searching only the directory where the first ZIP appeared would miss the copies.

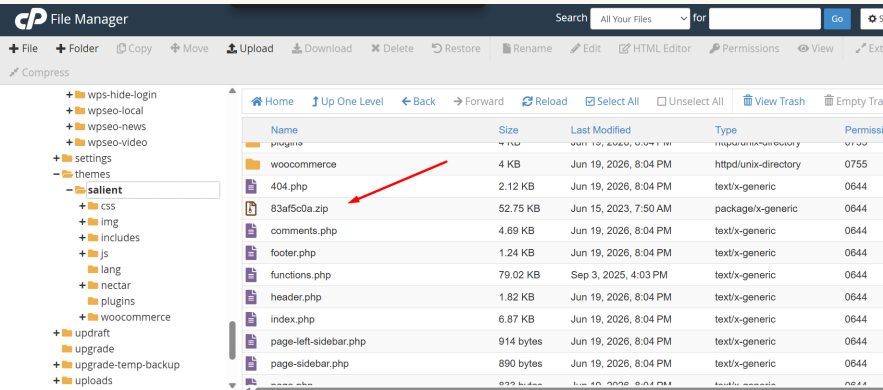


Figure 16.5. A second 83af5c0a.zip existed inside a theme directory. Its location supports the recovered archive-writing behavior and shows why a site-wide search was necessary.

Several names were generated from the victim's absolute WordPress path plus a short role label. The code took the first characters of an MD5 digest to make a stable name for that installation.

Role	Recovered formula	Name on this site
visible and hidden loader	first 8 of md5(ABSPATH + "ldr")	59e2e230
ZIP archive	first 8 of md5(ABSPATH + "zip")	83af5c0a
cache installer	first 8 of md5(ABSPATH + "inst")	b411933e
service worker	first 8 of md5(ABSPATH + "sw")	e777f187
database option	first 10 of md5(ABSPATH + "opt")	8aeb646ca5
theme marker suffix	first 8 of md5(ABSPATH + "th_m")	ccd02512

The absolute path is different on another server, so the resulting strings will also be different. The fake plugin slug can change independently. A responder must not copy 59e2e230, 83af5c0a, or a victim-specific database option name into a universal deletion list.

Hunt first for behavior, stable SC markers, decoded hashes, and relationships. Use generated names as incident-local pivots.

Why file dates could not establish the timeline

The recovered payload could copy plausible timestamps from existing WordPress PHP files or backdate artifacts by roughly a year. This behavior is commonly called **timestamping**.^[3]

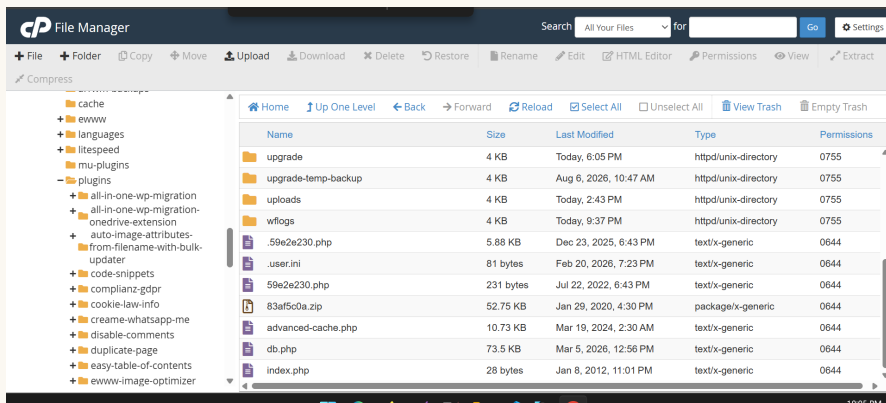


Figure 16.6. This second capture corroborates the loader, drop-in, and archive pattern. The displayed dates are presentation metadata, not reliable proof of when the compromise began.

The initial-access timeline therefore needs server evidence: raw access and error logs, WAF events, hosting-panel audit logs, SFTP or SSH logs, database audit records, and the earliest clean and infected backups. Filesystem dates can still guide triage, but they cannot carry the conclusion alone.

The database proved that persistence was not file-only

After the static file analysis, I ran read-only queries against the affected WordPress database. The examples below use `YOURPREFIX_`. Replace it with the exact `$table_prefix` from the site's `wp-config.php`. Never assume `wp_`, and do not paste one client's prefix into another site's query.

The first query searched only the stable names recovered from code and the related transient namespace:

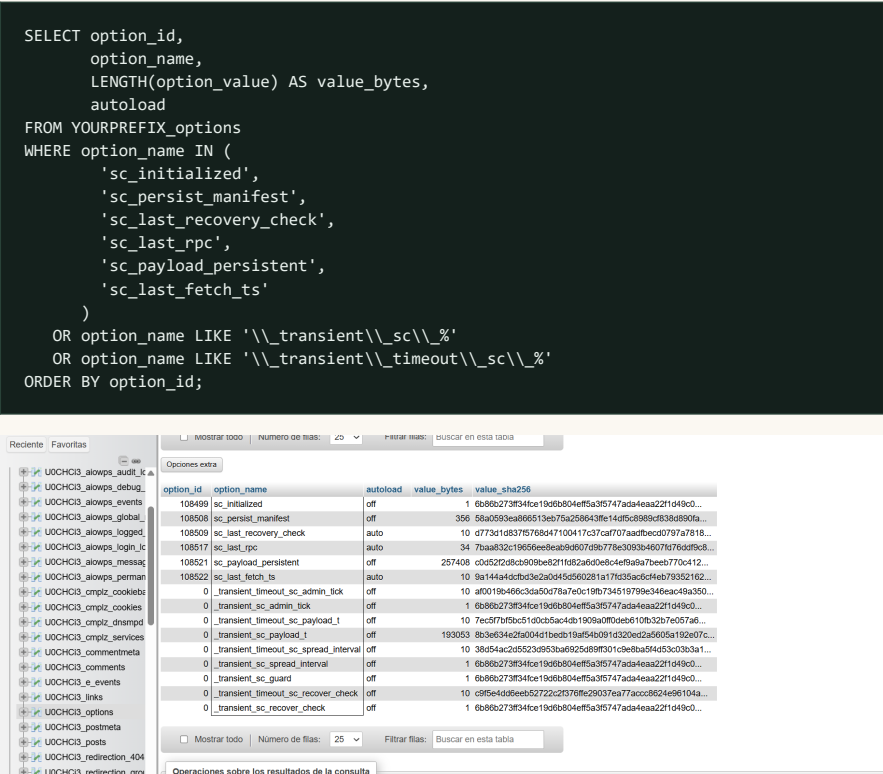


Figure 16.7. Six stable `sc_*` options and nine transient or timeout rows were returned. `sc_payload_persistent` contained 257,408 bytes, directly corroborating the database-payload behavior found in the PHP.

The stable option names came from the recovered code. They were not guessed from the screenshot and were not based only on size.

Confirmed option	Captured size	Interpreted role
sc_initialized	1 byte	initialization flag
sc_persist_manifest	356 bytes	persistence manifest
sc_last_recovery_check	10 bytes	recovery timing state
sc_last_rpc	34 bytes	last RPC state
sc_payload_persistent	257,408 bytes	persistent payload store
sc_last_fetch_ts	10 bytes	fetch timing state

Separate the transient payloads

WordPress normally stores a transient value and its expiration in separate option rows. Filtering the SC namespace made the payload and coordination rows easier to review:

```
SELECT option_id,
       option_name,
       LENGTH(option_value) AS value_bytes,
       autoload
FROM YOURPREFIX_options
WHERE option_name LIKE '\\_transient\\_sc\\_%'
      OR option_name LIKE '\\_transient\\_timeout\\_sc\\_%'
ORDER BY option_id;
```

Mostrando filas 0 - 8 (total de 9, La consulta tardó 0.0040 segundos.)

SELECT option_id, option_name, autoload, OCTET_LENGTH(option_value) AS value_bytes FROM U0CHC13_options WHERE option_name LIKE '_transient_sc_%' OR option_name LIKE '_transient_timeout_sc_%' ORDER BY value_bytes DESC;

Perfilando [[Editar en línea](#)] [[Explicar SQL](#)] [[Crear código PHP](#)] [[Actualizar](#)]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas:

Opciones extra

option_id	option_name	autoload	value_bytes	46
0	_transient_sc_payload_t	off	193053	
0	_transient_timeout_sc_admin_tick	off	10	
0	_transient_timeout_sc_payload_t	off	10	
0	_transient_timeout_sc_spread_interval	off	10	
0	_transient_timeout_sc_recover_check	off	10	
0	_transient_sc_admin_tick	off	1	
0	_transient_sc_spread_interval	off	1	
0	_transient_sc_guard	off	1	
0	_transient_sc_recover_check	off	1	

☐ Mostrar todo | Número de filas: 25 | Filtrar filas:

Consola

Figure 16.8. The transient-specific result included administrator-tick, payload, spread, guard, and recovery records. `_transient_sc_payload_t` held 193,053 bytes.

This matters because "clear the cache" is not an adequate database response. A malware payload stored through WordPress's transient mechanism is still malware state and must be exported, attributed, and removed with its timeout row during the coordinated cleanup.

Test the known cron hook without overstating a negative result

The recovered code registered a WordPress cron hook named `sc_cron_fetch`. I searched the serialized cron option for that exact string:

```
SELECT option_id,
       LENGTH(option_value) AS value_bytes,
       LOCATE('sc_cron_fetch', option_value) AS hook_position
FROM YOURPREFIX_options
WHERE option_name = 'cron';
```

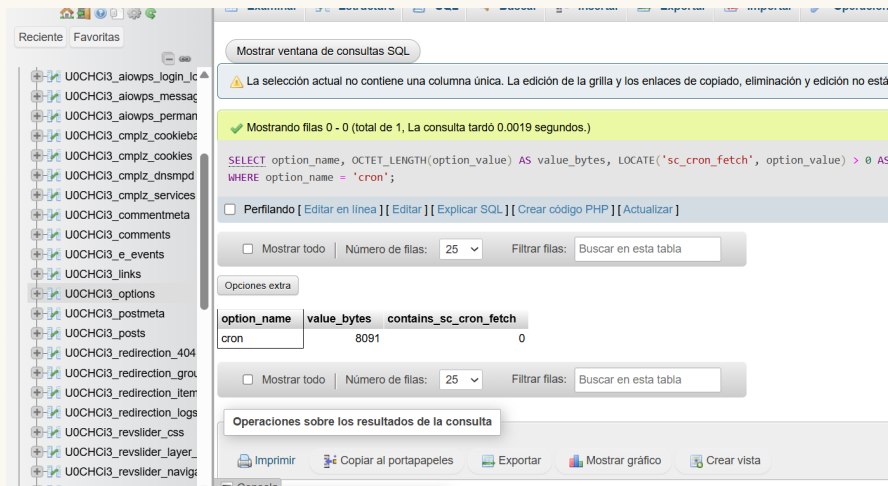


Figure 16.9. The `cron` value was 8,091 bytes, but `sc_cron_fetch` was absent at the time of this query.

This is a bounded negative finding. It does not prove that the hook was never registered, that another hook was not used, or that scheduling behavior in the recovered code did not exist. It records only what this database contained at one moment.

How the three hexadecimal option candidates were found

The names `60909f14f4a5`, `ce9ccf62d5c0`, and `c9fe246a17` did not come from a universal IOC list. They appeared during a review of the site's largest option values. The first two also had option IDs interleaved with confirmed SC rows, which made them high-priority candidates for export and offline inspection.

The discovery query was a prioritization tool:

```
SELECT option_id,
       option_name,
       LENGTH(option_value) AS value_bytes,
       autoload
FROM YOURPREFIX_options
ORDER BY LENGTH(option_value) DESC
LIMIT 100;
```

An optional filter can narrow the review to generated-looking names, but it still cannot establish maliciousness:

The screenshot shows a SQL query in a dark-themed editor and its results in a light-themed database management tool.

SQL Query:

```
SELECT option_id,
       option_name,
       LENGTH(option_value) AS value_bytes,
       autoload
FROM YOURPREFIX_options
WHERE option_name REGEXP '^[0-9a-f]{10,16}$'
ORDER BY option_id;
```

Database Management Tool Interface:

- Buttons: Reciente, Favoritas, Opciones extra.
- Actions: [Editar en línea], [Editar], [Explicar SQL], [Crear código PHP], [Actualizar]
- Table Headers: option_id, option_name, autoload, value_bytes, value_sha256
- Table Content (Partial):

option_id	option_name	autoload	value_bytes	value_sha256
112326	WPML	auto-off	2832522	d1e0a68f69f0830e06597aa159a8b941d5ed34945035930...
0	site_transient_feed_c117b5738b3c35bd8c391cda1f2b...	off	695919	8dce808a193d386a708e24599ad490b15990020f38defa...
0	site_transient_feed_69c333da78805ef0abe98576ff...	off	272116	484532c82aef070c0ded61c37ec83005d776c2c0434aea7452...
108500	60909f14f4a5	off	261136	52790de1f944dc974c521a6762fa8e208077bda1b1ee457bc5...
108521	sc_payload_persistent	off	257408	c0d522d8cb00be211d32a0d0ebcf9a9a7b6eb770ca12...
438	rs-library	off	214293	1ed5a23924a706c3b8fa237c72f81ec2dc0f0a598b07019f0...
0	_transient_sc_payload_1	off	193053	8b3e634e2ba004d1bcb19a5f4b091d320ed2a560a192e07c...
442	rs-templates	off	165960	83215130c88829da2b5c2dad7615c2db514542abba4ef54...
108508	cd9c62c250	off	151112	9a9e4c1985cc0ca071b4c2c23159a67ef6b0e89b911aef1...
112350	WPML_db_checksums_cache	auto	96569	c1f7058c3c3ba13a128f7c1f97a29a887aa465564867892...
0	c9fe246a17	off	76112	743d94433d3403403d8b3c3da9a0a20cc0cb11333e0350a8b...
1382	vs_menu_editor	auto	68057	92d9a97fda66c7972ef6c556e087a8135aefba1458a904d2ac...
0	site_transient_available_translations	off	49205	aa29a10a3465a294cdd1b3a74030c3d746698e8db6923083...
287	eltdf_options_laurent	auto	36438	3509452553c3b161ea79d3b4842ec0abd23966663d468c25...
435	revslider-addons	auto	36227	7b10d868cec23219e523b16f781162d2e90e6559f5ea9302...
0	_transient_cmpiz_cookies	off	33370	1867032ef796ae6459c1b6515340c1b4a08df4ef15af69011...
455	elementor_log	off	26605	7fa0baca26050a39c50f95e164928e17ec385cab76aecdb08...
112097	_transient_laurent_social_icons_collections	off	24939	456b993592c009c32b7a1438ba9b01cd2bb662e7ee8f300c91...
0	_transient_wp_core_block_css_files	on	22750	d380e0195964c0f10a2a216cc1bca5385f890c20f6979b0...

Figure 16.10. Confirmed SC rows appeared beside 60909f14f4a5 at 261,136 bytes, ce9ccf62d5c0 at 151,112 bytes, and c9fe246a17 at 76,112 bytes. Name shape, size, and neighboring IDs prioritize review; they do not prove that the three rows are malicious.

The calculated 8aeb646ca5 option name from the recovered path formula was not returned in this capture. I did not assume that one of the observed candidates was equivalent to it.

Before deleting any generated-looking option on another site:

1. export the complete row to a protected evidence location;
2. search the preserved PHP for the exact option name;
3. inspect nearby option IDs and creation context;
4. determine whether a known legitimate plugin owns it;
5. decode a copy offline without deserializing or executing PHP objects; and
6. compare decoded size and SHA-256 with confirmed incident payloads.

Large values and hexadecimal names are clues. Legitimate plugins also store large caches, CSS, templates, and generated identifiers.

Check the recovered administrator pattern, then check every administrator

The payload could create a username beginning `admin_`, `adm_`, `administrator_`, or `backup_`, followed by ten lowercase hexadecimal characters. This read-only query tested that exact pattern:

```
SELECT ID, user_login, user_email, user_registered
FROM YOURPREFIX_users
WHERE user_login REGEXP
'^^(admin|adm|administrator|backup)_[0-9a-f]{10}$';
```

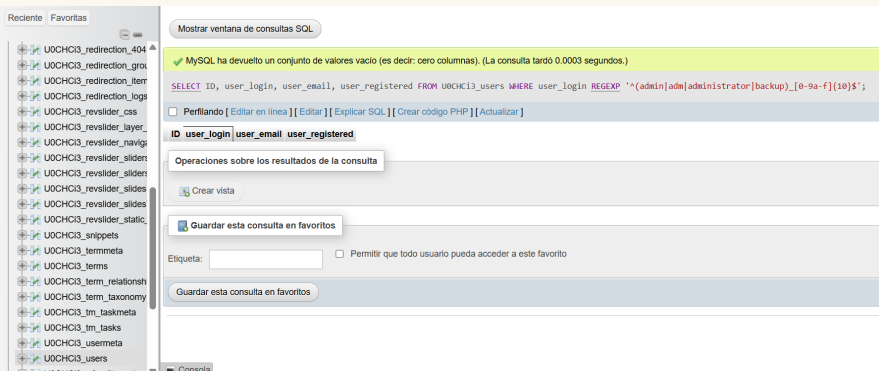


Figure 16.11. No account matched the generated username pattern in this capture. The negative result did not clear the user table.

An attacker can use a normal-looking username, change the generator, or remove an account after creating sessions. The complete administrator inventory must therefore be reviewed through capabilities metadata as well as `wp-admin`:

```
SELECT u.ID,
       u.user_login,
       u.user_email,
       u.user_registered,
       um.meta_value AS capabilities
FROM YOURPREFIX_users AS u
JOIN YOURPREFIX_usermeta AS um
  ON um.user_id = u.ID
WHERE um.meta_key = 'YOURPREFIX_capabilities'
      AND um.meta_value LIKE '%administrator%'
ORDER BY u.ID;
```

The prefix inside `meta_key` must match the site too.

What the decoded payload could do

Persistence was only one part of the compromise. Static analysis established the following capabilities for this recovered sample set.

Resolve command and control through Ethereum

The payload rotated through public Ethereum JSON-RPC providers and issued `eth_call` with selector `0x3bc5de30` against embedded contract addresses. It parsed the response to recover an XOR key and encrypted C2 URLs.^[4]

This design used legitimate public infrastructure as a dead-drop resolver. An RPC provider's presence in a log is supporting context, not proof that the provider itself is malicious. The stronger combination is the selector, contract address, request pattern, recovered code, and related host behavior.

Steal credentials and retain administrator access

The code could record a successful administrator's submitted plaintext password, create and conceal an administrator, and generate WordPress authentication cookies for existing administrators. It also suppressed selected notifications that might alert the owner.

Changing one WordPress password would not revoke all of those paths. Recovery had to include every administrator, all sessions, WordPress salts, hosting and deployment credentials, the database password, and secrets available to the compromised runtime.

Use the administrator browser

The service-worker template could capture login submissions and use an authenticated administrator context to attempt plugin reinstallation. The `Service-Worker-Allowed: /` header can permit a worker script to control a scope above its own directory when the browser accepts that scope.^[5]

This expanded the incident boundary beyond the server. Administrator browsers and profiles needed their service workers and site data cleared after the server was clean and before new credentials were used.

Spread to sibling WordPress sites

The payload searched parent and common shared-hosting directories for writable WordPress installations. It could place itself in a sibling site's MU-plugin or ordinary plugin directory and, when needed, read that site's configuration to locate its database and active-plugin state.

The same malware on several sites under one operating-system user does not prove that each site was exploited separately. It may show lateral spread from one compromised account boundary. Every writable sibling site therefore belonged in scope.

What the samples did not prove

The recovered code proved persistence, credential theft, browser capability, C2 resolution, defense evasion, and sibling-site spread. It did not reveal the first vulnerability or credential used to compromise the original site.

Possible routes included a vulnerable extension, stolen WordPress or hosting credentials, compromised SFTP or SSH access, a file-manager flaw, a poisoned deployment, or spread from another site. Those are hypotheses until historical logs or backups support one of them.

The analysis also did not prove that every coded capability ran successfully. Static code can establish that a function exists and can be reached under its conditions. A network capture, database record, browser artifact, or log is needed to prove a particular execution on a particular date.

Remove the persistence mesh in one maintenance window

Deleting components one at a time on a live site gives the remaining nodes a chance to restore them. The safe sequence is coordinated.

1. Preserve and contain

1. Export a filesystem snapshot, database dump, and relevant logs outside the compromised hosting account.
2. Record hashes and preserve representative malicious files as non-executable evidence.
3. Put the virtual host behind a static maintenance response or isolate it.
4. Stop or restart the site's PHP workers as part of the maintenance window so cached `.user.ini`, OPcache, and process-resident shared memory do not keep running.
5. Avoid signing in through an ordinary administrator browser until its service-worker state can be cleared.
6. Identify every site writable by the same operating-system account.

2. Remove every file and memory recovery source

1. Remove the malicious ordinary plugin and MU-plugin copies.
2. Remove the visible and hidden loaders.
3. Remove unauthorized `auto_prepend_file` settings from every applicable `.user.ini`, `.htaccess`, PHP-FPM pool, and virtual-host configuration.
4. Replace malicious `advanced-cache.php` and `db.php` from the legitimate product that owns them, or remove them only after verifying that the site does not need those drop-ins.
5. Restore the affected theme from a trusted source or remove the exact marked block after a careful diff. Inspect inactive themes too.
6. Find and remove connected ZIP archives from `wp-content`, `uploads`, and theme directories.
7. Find hashed installers, guards, service-worker scripts, and unexpected PHP under writable cache or upload locations. Validate content before removal.
8. Rebuild WordPress core, plugins, and themes from verified vendor packages.
9. Restart the isolated PHP environment to destroy the implant's process memory and load the known-clean code.

3. Export and remove confirmed database persistence

Export the matched rows first. Then delete the confirmed stable keys and SC transient namespace using the site's actual prefix. A transaction gives the operator a final review point when the table engine supports it:

```

START TRANSACTION;

SELECT option_id, option_name, LENGTH(option_value) AS value_bytes
FROM YOURPREFIX_options
WHERE option_name IN (
    'sc_initialized',
    'sc_persist_manifest',
    'sc_last_recovery_check',
    'sc_last_rpc',
    'sc_payload_persistent',
    'sc_last_fetch_ts'
)
OR option_name LIKE '\\_transient\\_sc\\_%'
OR option_name LIKE '\\_transient\\_timeout\\_sc\\_%';

DELETE FROM YOURPREFIX_options
WHERE option_name IN (
    'sc_initialized',
    'sc_persist_manifest',
    'sc_last_recovery_check',
    'sc_last_rpc',
    'sc_payload_persistent',
    'sc_last_fetch_ts'
)
OR option_name LIKE '\\_transient\\_sc\\_%'
OR option_name LIKE '\\_transient\\_timeout\\_sc\\_%';

SELECT ROW_COUNT() AS deleted_rows;

COMMIT;

```

Do not add 60909f14f4a5, ce9ccf62d5c0, c9fe246a17, or any other hexadecimal option to this deletion solely because it appeared in Figure 16.10. A confirmed site-specific row should be deleted by its exact name only after ownership and offline-content validation.

Review the serialized cron value through WordPress or WP-CLI where possible instead of manually editing serialized data. Remove a confirmed malicious hook through the API that created it, or with a carefully tested incident-specific method.^[6]

4. Revoke server, WordPress, and browser access

1. Remove every unauthorized administrator and verify the remaining account owners directly.
2. Destroy all WordPress sessions and rotate authentication salts.
3. Rotate hosting, control-panel, SSH, SFTP, FTP, database, deployment, SMTP, API, and WordPress credentials exposed to the site.

4. Review delegate access, recovery email addresses, API tokens, scheduled deployments, and support accounts that can change the site from outside WordPress.
5. On every administrator device and browser profile, unregister the site's service worker and clear cookies, Cache Storage, IndexedDB, local storage, and other site data.
6. Use new credentials only after the server and browser cleanup are complete.

5. Verify by trying to trigger recurrence

A clean scan is not the end of this case. Verification should test the paths that previously caused regeneration.

- Start fresh PHP workers and request public and administrative pages.
- Recheck the ordinary plugin, MU-plugin, drop-in, theme, loader, ZIP, upload, cache, and configuration paths.
- Rerun the stable database-marker and transient queries.
- Recheck administrator capabilities, sessions, and scheduled events.
- Inspect administrator browsers for a reappearing service worker.
- Monitor new or changed files, outbound requests, and sibling sites through the former recurrence window.
- Compare the live site with a documented known-good baseline.

If one artifact returns, preserve the new timestamp and writer evidence before deleting it again. The returning file is now a sensor pointing toward a surviving writer.

High-value hunting indicators

These content indicators are stronger than the fake plugin name:

```
SC_ADV_BEGIN:4.0.3:
SC_DB_BEGIN:4.0.3:
SC_TH_BEGIN:4.0.3:
sc_payload_persistent
sc_persist_manifest
sc_cron_fetch
0x3bc5de30
```

Important recovered SHA-256 values include:

Recovered object	SHA-256
decoded core payload	8284d68274c0475118732110680e38d9da27b4173997c0db0d9f6e5283109554
hidden loader	5e0634b36af4fab6931e3efc120e5baee1953f32f14a516cbc4c927786d4309c
visible wrapper	96d36f0432295b0a833e6d835c8f3100f900ace2f08f0584ea0d2a83e7df8b82
malicious advanced-cache.php	1bd22238cd2679cd8dcdd8e1d391339ae07f009b67277929951d18d75d55cf0
malicious db.php	a3c6fcd1df2c1fec977427f056aa45852f7ae7a9f63ff79813057e451eed6c3
infected theme file	b516bd7eb514b80555fc3a04ac30405e52f128c6d08da3baf03e74f1ad2fe1fb
malicious .user.ini	ac7c266d12c6d3d979594b4d74c545ecf2546ac704f2ac07bde1276010747c0d
service-worker template	7bafee00a010ed3db3521a89e8f5d50d2f677204c56995e99c7603768a9f0bd2

A matching hash is high-confidence evidence for that exact file. A different hash does not clear a site because wrappers, paths, aliases, and updated payloads can change.

The durable lesson

At first, this incident looked like a plugin that would not stay deleted. The complete evidence showed something more useful: PHP configuration, WordPress bootstrap files, an ordinary plugin, a must-use plugin, a theme, archives, the database, process memory, scheduled behavior, administrator sessions, and a browser could all participate in persistence.

The temporary observation that renaming wp-content interrupted regeneration was an important clue. It broke several calculated and hard-coded paths at the same time. It was not the permanent fix.

The permanent response was to map the graph, decode without execution, confirm the database separately, remove all recovery nodes under containment, revoke every exposed access path, clean administrator browsers, inspect sibling sites, and verify the former triggers.

That is the difference between deleting malware and ending an incident.

Source notes

- [1]: PHP manual, ".user.ini files" and `auto_prepend_file`:
<https://www.php.net/manual/en/configuration.file.per-user.php> and
<https://www.php.net/manual/en/ini.core.php>.
- [2]: WordPress Developer Resources, "Must Use Plugins," drop-ins, and WordPress configuration:
<https://developer.wordpress.org/advanced-administration/plugins/mu-plugins/>,
https://developer.wordpress.org/reference/functions/_get_dropins/, and
<https://developer.wordpress.org/advanced-administration/wordpress/wp-config/>.
- [3]: MITRE ATT&CK, "Timestamp: T1070.006": <https://attack.mitre.org/techniques/T1070/006/>.
- [4]: Ethereum developer documentation, JSON-RPC `eth_call`:
<https://ethereum.org/developers/docs/apis/json-rpc/>.
- [5]: MDN Web Docs, `Service-Worker-Allowed`:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Service-Worker-Allowed>.
- [6]: WordPress Developer Resources, `wp_schedule_event()` and WP-CLI cron event commands:
https://developer.wordpress.org/reference/functions/wp_schedule_event/ and
<https://developer.wordpress.org/cli/commands/cron/event/list/>.
- The complete public technical report for this case is "SC 4.0.3 Self-Healing WordPress Malware That Rebuilds Itself": <https://www.mdlabel.com/malware-research/sc-403-self-healing-wordpress-malware/>.

Field Checklist

Use this checklist as a working companion to the case study. It is deliberately short; the investigation explains why each step exists.

Preserve and contain

- [] Export the filesystem, database, and relevant logs outside the affected hosting account.
- [] Hash representative suspicious files and preserve them as non-executable evidence.
- [] Record the visible symptom, affected devices, user roles, URLs, and recurrence trigger.
- [] Isolate the site or serve a static maintenance response.
- [] Identify every website writable by the same operating-system account.
- [] Avoid ordinary administrator browsing until browser persistence has been checked.

Map the persistence graph

- [] Inspect `.user.ini`, `.htaccess`, PHP-FPM, and virtual-host configuration for an unauthorized `auto_prepend_file` or equivalent early loader.
- [] Compare ordinary plugins, must-use plugins, drop-ins, active and inactive themes, uploads, cache directories, and unexpected archives.
- [] Search preserved files for stable content markers and hashes, not only the fake plugin name.
- [] Record what each component can execute, restore, download, hide, or recreate.
- [] Treat timestamps as triage clues unless logs or backups corroborate them.

Decode safely

- [] Work from copies in an isolated directory.
- [] Normalize only literal transformations you can explain.
- [] Decode Base64 and decompress data without `eval`, PHP includes, network requests, or browser execution.
- [] Save decoded output with a non-executable extension.
- [] Compare byte counts and SHA-256 values across recovered payload copies.

Validate the database

- [] Read the exact table prefix from `wp-config.php`; never assume `wp_`.
- [] Query stable option names recovered from the code.
- [] Inspect related transient values and their timeout rows.
- [] Search the serialized cron option for recovered hook names without manually editing serialized data.
- [] Review the largest option values and generated-looking names as leads, not proof.
- [] Export each candidate row and identify legitimate plugin ownership before deletion.
- [] Inventory every administrator through capabilities metadata, not only the wp-admin Users screen.

Remove and revoke

- [] Stop or restart PHP workers during the maintenance window.
- [] Remove confirmed loaders, payload copies, malicious drop-ins, injected theme code, archives, cached installers, and unauthorized configuration.
- [] Rebuild WordPress core, plugins, and themes from trusted packages.
- [] Export and remove confirmed malicious options, transients, and scheduled hooks.
- [] Remove unauthorized administrators and destroy all WordPress sessions.
- [] Rotate WordPress salts plus hosting, SSH, SFTP, database, deployment, SMTP, API, and administrator credentials exposed to the site.

- [] Clear service workers, cookies, Cache Storage, IndexedDB, local storage, and site data from administrator browser profiles.

Verify

- [] Start fresh PHP workers and request both public and administrative pages.
- [] Repeat the file, database, user, cron, browser, and sibling-site checks.
- [] Trigger the actions that previously caused regeneration.
- [] Compare the recovered site with a recorded known-good baseline.
- [] Monitor new files, changed files, outbound requests, administrator actions, and scheduled events through the former recurrence window.
- [] If an artifact returns, preserve its new timestamp and writer evidence before deleting it again.

Continue with the Complete Book

This case study is adapted from Chapter 16 of *WordPress Malware Removal for Developers & Site Owners* by MD Pabel.

The complete 278-page book contains 17 chapters and 81 real screenshots and figures. It covers the full recovery path: confirming a compromise, preparing before cleanup, finding the complete scope, cleaning WordPress files and the database, inspecting hosting and DNS, removing redirects and SEO spam, identifying fake plugins and hidden administrators, investigating WooCommerce skimmers, recovering suspended and blacklisted sites, stopping reinfection, and monitoring the known-good baseline.

Read the complete book

[Get *WordPress Malware Removal for Developers & Site Owners* on Leanpub.](#)

About the author

MD Pabel is an independent WordPress security specialist and developer working in WordPress security since 2018. His writing turns first-hand cleanup and malware investigations into repeatable methods that developers, freelancers, and site owners can use without confusing a scanner result or a suspicious filename with complete proof.